

Mp10 — Signature Helper

Capturing patient signatures on a Topaz pad from Mp10 Web

Structured Systems · www.structuredsystems.com

13 August 2026

Contents

1	About this guide	3
1.1	The short version	3
1.2	Why there is a separate program at all	3
2	Using it — front desk	4
2.1	What it does not do	4
3	When something is wrong	5
4	Installing it — IT task	6
4.1	It is not a service	6
4.2	What to copy	6
4.3	Settings	6
4.4	Telling MpSigSrv which pad	7
4.5	The consent text	7
4.6	Check it	8
4.7	Updating a workstation later	8
5	Notes for the curious	10
5.1	Is it safe to have a program listening on the workstation?	10
5.2	Why does the operator code travel with each capture?	10
5.3	What happens if a capture hangs	10
5.4	Does an https site really reach a http://127.0.0.1 helper?	10

1 About this guide

Mp10 Web takes patient signatures on a **Topaz signature pad**, and it does that through a small companion program called **MpSigSrv** that runs on the workstation the pad is plugged into.

There are two audiences here. **Using it** is for the person at the front desk and is two pages. Everything from *Installing it* onward is an **IT task** and is where the setup actually lives.

If you have used **MpPrintSrv**, the print helper, this will look familiar — same idea, same loopback design. **One thing is different and it matters: MpPrintSrv is a Windows service; MpSigSrv is not and cannot be.** See [It is not a service](#).

A note on paths. This guide says *the Mp10 folder* for wherever the Mp10 programs are installed on that workstation. It is `C:\Mp10` on a typical installation, but it is a per-site choice — check where `Patients10.exe` lives on the machine in front of you and use that.

1.1 The short version

Program	MpSigSrv.exe, in the Mp10 program folder (<code>C:\Mp10</code> on a typical install)
Runs on	the workstation with the pad, in the signed-in user's session
Started by	MpSigSrv.exe -listen, at logon
Port	6266 (MpPrintSrv is 6265; they coexist)
Is it running?	open http://127.0.0.1:6266/ping in the browser on that PC
Settings	[RDD] and [SIGN] in Mp10.ini, beside the exe
Logs	MpSigSrv.log and Topaz.log, beside the exe

1.2 Why there is a separate program at all

The pad faces the **patient**. The consent text, the choice they make and the Next control all belong on the pad's own small screen, exactly as the Mp10 desktop puts them there — the browser is the *operator's* screen, and the patient never looks at it.

A browser cannot drive that screen. Beyond that, the consent text is allowed to contain expressions the Mp10 desktop evaluates at the moment of signing (`^email` is the one this practice uses, and `~...~` can hold any Harbour expression). Reproducing that in JavaScript is not merely awkward, it is not possible. So MpSigSrv runs **the desktop's own signing code**, and a patient signing from the web sees exactly what a patient signing from the desktop sees.

MpSigSrv also **stores the signature itself**, through the same statement the desktop uses. Nothing about the signature travels back through the browser.

2 Using it — front desk

1. Open the encounter in Mp10 Web and press **Capture signature**.
2. The browser shows you the consent the patient is being shown — this is your copy to follow, not theirs.
3. Press **Capture on pad**.
4. **Hand the pad to the patient.** They read the text, page through it with the arrow, tap their choice if the consent offers one, and sign.
5. A window appears on your screen showing the signature they gave. Press the pen button to keep it, or Cancel to discard it and start again.
6. The browser says “*Signature captured on the pad and saved.*”

The options shown in the browser are greyed out on purpose. **The patient chooses on the pad**; a choice made on your screen would be a consent they never gave.

2.1 What it does not do

- **Patient-record (HIPAA) signatures** are not on the pad from the web yet. Only encounter signatures are. The desktop still does the patient record.
 - **It is Windows-only**, because the pad is. A Mac or a tablet cannot capture from a pad at all — attach the signature from the record instead.
-

3 When something is wrong

What you see	What it means	What to do
<i>The signature helper is not answering on this workstation</i>	Either MpSigSrv is not running here, or its [SIGN] ORIGIN does not list this site. A browser cannot tell those apart.	IT task — see Check it . Check ORIGIN first ; it is the more common of the two.
<i>No operator code for the signed-in user</i>	This Mp10 Web account has no Operator set. The signature would not record who witnessed it, so it is refused.	Set an Operator on that account in the desktop Admin → Users .
<i>The consent text for this signature cannot be read</i>	sys_registry points at a consent file the server cannot open.	IT task — see The consent text .
<i>The signature pad could not be activated</i>	MpSigSrv could not talk to the pad.	Check the pad is plugged in and [Tablet] TabletComPort in C:\Windows\SigPlus.ini matches its COM port. See Telling MpSigSrv which pad .
The pad shows nothing, or the wrong size text	The pad geometry is wrong.	
<i>The capture did not finish within 300 seconds</i>	Nobody completed the signature and the helper gave up.	Press Capture on pad again. If it keeps happening, restart MpSigSrv.

Two log files sit beside the exe and between them they answer most of this:

- **MpSigSrv.log** — what the helper did: what it connected to, whether it is listening, each capture requested, the option picked, and every refusal.
- **Topaz.log** — what the *pad* reported: its model number and its screen size. This is the file to read when the layout looks wrong.

4 Installing it — IT task

4.1 It is not a service

MpSigSrv cannot be installed as a Windows service, and there is no -i switch. It hosts the Topaz ActiveX control and opens a window; a service runs in session 0, which has no interactive desktop, so neither would work.

It runs **in the signed-in operator's own session**, started at logon. That is a real difference from MpPrintSrv, which is a service — if you set MpSigSrv up the way you set that up, it will not work.

Practically, that means one of:

- a shortcut to `MpSigSrv.exe -listen` in the user's **Startup** folder (`shell:startup`), or
- **Task Scheduler**, trigger *At log on*, action `<Mp10 folder>\MpSigSrv.exe`, arguments `-listen`, and **“Run only when user is logged on”** — not the “whether or not” option, which puts it back in session 0.

It shows no window while it waits. To stop it, end `MpSigSrv.exe` in Task Manager.

4.2 What to copy

Onto each workstation that has a pad:

File	What it is
<code>MpSigSrv.exe</code>	the helper
<code>Mp10.ini</code>	settings — copy <code>Mp10.ini.example</code> and edit
<code>ace32.dll</code> , <code>axcws32.dll</code>	the 32-bit Advantage client, already present on any PC running the Mp10 desktop apps
<code>SigPlus.ocx</code>	Topaz's control — installed and registered by Topaz's own SigPlus installer, not copied by hand

MpSigSrv is a 32-bit program and uses the **same 32-bit ACE client the desktop applications use**. This is the opposite of Mp10 Web's server, which needs the 64-bit client — do not confuse the two.

`SigPlus.ocx` comes from Topaz. Install **SigPlus** (not SigWeb — that is the browser component and is not used here) from <https://www.topazsystems.com>, which registers the control and writes `C:\Windows\SigPlus.ini`.

4.3 Settings

`Mp10.ini`, beside the exe. Two sections matter.

```
[RDD]
RDD-VERSION=REMOTE
PATH=\\adssserver\adsdata\sfi\
user=AutoTasks
password=mp10

[SIGN]
PORT=6266
ORIGIN=https://mp10web.example.org
```

PORT — 6266 unless something else on the workstation wants it. It is also compiled into Mp10 Web as the default, so changing it here means changing it there too (`VITE_SIGN_URL`). MpPrintSrv's 6265 is a different program on a different port; both can run at once.

ORIGIN — the address staff open Mp10 Web at, exactly as the browser sends it: scheme and host, no trailing slash, and the port when it is not 80 or 443. Several may be listed, comma separated.

This is the whole of the access control, and it is the thing most likely to be wrong.

With nothing here, every browser request is refused — and a browser cannot tell a refused origin from a helper that is not running. Both arrive as the same failure. If Mp10 Web says the helper is not answering, check this line before anything else. `MpSigSrv.log` records the refusal by name, which is how you tell the two apart.

[RDD] **user** is the account MpSigSrv connects to the dictionary as. **It is not the person who witnesses the signature** — that is the operator code Mp10 Web sends with each capture. Do not put an operator code here: an operator code is not an ADS login and the server answers 7078.

4.4 Telling MpSigSrv which pad

Two files decide this and they do different jobs.

`C:\Windows\SigPlus.ini`, written by Topaz's installer, says how to reach the pad:

```
[Tablet]
TabletComPort=9
TabletModel=LCD4X5SE
```

The pads at this practice are **serial** devices reached through a USB-to-serial adapter, so they appear as a COM port rather than as a USB device. Find the number in Device Manager under **Ports (COM & LPT)** — the entry whose name or serial number contains TOPAZ.

The consent JSON (see below) says how big the screen is, through its `TabletModel` key:

```
{ "TabletModel": 1, ... }
```

1 means the 320×240 screen; anything else means 240×64.

This is not optional and it is not guesswork. Measured on this practice's pad: `TabletModelNumber()` — the call that asks the device what it is — returned 0 from one program and 58 from another, and `GetLCDSize()` only reports whatever layout was last set, not the physical panel. **The pad does not reliably identify itself.** The value in the JSON is what actually sets the geometry. Get it wrong and the consent is laid out for the wrong screen.

To see what your pad reported, take one signature and read `Topaz.log`:

```
TabletModelNumber() 58   xMax   yMax
TTOPAZSIGNATURE:INITPAD  nTabletModel 1   xMax 240   yMax 64
nTabletModel 1   xMax 320   yMax 240
```

The last line is the geometry in force.

4.5 The consent text

The text, the options and the font all come from `sys_registry`, from the **same row the desktop reads** — so editing it in Admin changes both the desktop and the web at once. For encounters that row is TOPAZ / ENCOUNTER-SIGNATURE-DISPLAYMESSAGE.

Its value is either the text itself, or **the path to a file holding it**. This practice uses the file form:

```
\\server\\share\\EncounterSignature-Text.txt
```

and that file holds JSON:

```
{
  "TabletModel": 1,
  "NewPageForOptions": false,
```

```

    "NewPageForSignature": false,
    "FontSize": 15,
    "FontName": "Arial",
    "Options": [
      { "id": 1, "Option": "Acepto mis resultados por email..." },
      { "id": 2, "Option": "No acepto el envío..." }
    ],
    "ShowTextOnPad": "Autorizo a ... Mi email es: ^email "
  }
}

```

^email is replaced with the patient's address at the moment of signing.

If the path is a UNC share, the account running MpSigSrv must be able to read it. A path that works when you are logged in can still fail for a different account. When the file cannot be read, Mp10 Web refuses the capture and names the path — it does not show the patient a file path as though it were a consent, which is what it used to do.

4.6 Check it

Three checks, in this order. Do them on the workstation with the pad.

1. Is it running and listening? Open this in the browser:

`http://127.0.0.1:6266/ping`

You should get:

```
{"ok":true,"app":"MpSigSrv","build":"2026-08-11 20:11:26"}
```

Nothing at all means it is not running — start `MpSigSrv.exe -listen` and look at `MpSigSrv.log`.

2. Is it configured the way you think?

`http://127.0.0.1:6266/status`

```

{"ok":true,"app":"MpSigSrv","port":6266,"origins":["https://mp10web.example.org"],
 "service":"not a service - runs in the operator's session",
 "captured":0,"failed":0,"wedged":false}

```

`"origins":[]` is the failure to look for. It means no [SIGN] ORIGIN, and every request from a browser will be refused however healthy the rest looks.

3. Does the pad actually work? Without a browser in the way, from a command prompt in the Mp10 program folder:

```
MpSigSrv.exe -enc=EN26-00000123 -operator=RCB
```

Use a real encounter number and a real operator code. The pad should show the consent; sign it and press the pen button. This is the check to run when the browser says something vague, because it takes the browser, the origin allowlist and the network out of the picture and leaves only the pad and the dictionary.

`-operator=` is **required**. It is stored on the signature as the record of who witnessed it, and there is no safe default — the helper refuses without one rather than inventing a value.

MpSigSrv is a windowless program, so it prints nothing to the command prompt. Read `MpSigSrv.log`.

Finally, in Mp10 Web: open an encounter and press **Capture signature**.

4.7 Updating a workstation later

Stop it first — a running copy holds the exe open and cannot be overwritten:

1. End `MpSigSrv.exe` in Task Manager.

2. Copy the new `MpSigSrv.exe` over the old one.
3. Start it again, or log off and back on if it is started at logon.

The build a workstation is running is reported by `/ping` and `/status`, which is how you confirm the copy actually took.

5 Notes for the curious

5.1 Is it safe to have a program listening on the workstation?

It binds **127.0.0.1** only — nothing off that machine can reach it, and no firewall rule is needed. On top of that it refuses any browser request whose origin is not in `[SIGN] ORIGIN`. Because a JSON POST is a *preflighted* cross-origin request, a page from any other site is stopped by the browser before its request is ever sent.

It does not check the Mp10 Web login token, and Mp10 Web deliberately does not send one — the helper has no use for it, and putting a session credential on that wire would only widen what a mistake could leak.

This does not defend against other software already running as that user on that PC. Neither does Topaz's own SigWeb, and that is the same trade the practice already accepted for signature capture.

5.2 Why does the operator code travel with each capture?

`patientsignatures.operator` records **who witnessed the signature**. The desktop gets that free, because the person signing in to the desktop *is* the dictionary user. MpSigSrv connects as a service account, so if it used the connection's identity every signature would be attributed to **AutoTasks**. That happened once, in testing, which is why the operator is now sent explicitly and a capture without one is refused.

5.3 What happens if a capture hangs

The helper answers `/ping` and `/status` throughout, because HTTP is handled on a separate thread from the pad. A capture that nobody finishes is abandoned after 300 seconds — long, deliberately, because that clock is measuring a patient reading a consent, not a computer doing work.

5.4 Does an https site really reach a http://127.0.0.1 helper?

Yes. Browsers treat loopback as a trustworthy origin and exempt it from mixed-content blocking. This is the same thing MpPrintSrv relies on.